Module III

MODULAR PROGRAMMING

Modular programming is the process of subdividing a computer program into separate sub-programs. It is a software design technique that emphasizes separating the functionality of a program into independent, interchangeable modules, such that each contains everything necessary to execute only one aspect of the desired functionality.

Advantages

- **Ease of Use :**This approach allows simplicity, as rather than focusing on the entire thousands and millions of lines code in one go, we can access it in the form of modules.
- **Programming errors are easy to detect:** Minimizes the risks of ending up with programming errors and also makes it easier to spot errors, if any.
- **Allows re-use of codes:** A program module is capable of being re-used in a program which minimizes the development of redundant codes
- **Improves manageability:** Having a program broken into smaller sub-programs allows for easier management.

FUNCTION

A function is a group of statements that together perform a task. Every Cprogram has at least one function, which is **main()**, and can have additional functions.

Types of Functions

There are two types of functions in C programming:

- Standard library functions
- User-defined functions

STANDARD LIBRARY FUNCTIONS

functions are defined in header files. For example,

- The printf() is a standard library function to send formatted output to the screen (display output on the screen). This function is defined in the stdio.h header file.
- The sqrt() function calculates the square root of a number. The function isdefined in the math.h header file.
- The strlen() function calculates the length of a given string. The function is defined in the string.h header file.

USER-DEFINED FUNCTIONS

Functions that we define ourselves to do certain specific task are referred as user-defined functions.

Advantages of user-defined functions

- The program will be easier to understand, maintain and debug.
- Reusable codes that can be used in other programs
- A large program can be divided into smaller modules. Hence, a large project canbe divided among many programmers.

Three parts of a user defined functions are:

1) Function Declaration or Prototype
2) Function Definition
3) Function Call

Function Declaration

A function prototype is simply the declaration of a function that specifies function's name, parameters and return type. It doesn't contain function body. A function prototype gives information to the compiler that the function may later be used in the program.

Syntax

Note:

- ✓ If function definition is written after main, then only we write prototypedeclaration in global declaration section
- ✓ If function definition is written above the main function then ,no need towrite prototype declaration

Function Definition

A function definition in C programming consists of a *function header* and a *function body*. Function header consist of return type,function name,arguments(parameters).

- **Return Type**: A function may return a value. The **return_type** is the data type ofthe value the function returns. Some functions perform the desired operations without returning a value. In this case, the return_type is the keyword **void**
- **Function Name**: The actual name of the function. The function name and the parameter list together constitute the function signature.
- **Parameters**: A parameter is like a placeholder. When a function is invoked, you pass a value to the parameter. Parameters are optional; that is, a function may contain no parameters.
- **Function Body**: The function body contains a collection of statements that define what the function does.

Syntax:

```
return_type function_name( argument list ) {
    body of the function
}
```

Function Call

When a program calls a function, the program control is transferred to the called function. A called function performs a defined task and when its return statement is executed or when its function-ending closing brace is reached, it returns the program control back to the main program.

Syntax:

```
functionName(parameter list);
```

Example:

```
int add(int,int,int);void main()
{
        int x=10,y=20,z=30,res;
        res = add(x,y,z);               // function call
        printf("Result=%d",res);
}
int add(int a, int b, int c)            // function definition
{
        int sum;
        sum = a+ b + c;
        return sum;                     //return statement
}
```

Return Statement

The return statement terminates the execution of a function and returns a value to the calling function. The program control is transferred to the calling function after the return statement.

In the above example, the value of the **sum** variable is returned to the main function. The **res** variable in the main() function is assigned this value.

Formal and Actual Parameters

There are different ways in which parameters can be passed into and out of functions. Let us assume that a function *B()* is called from another function *A()*. In this case *A* is called the ***"caller function"*** and *B* is called the ***"called function or callee function"***. Also, the arguments which *A* sends to *B* are called *actual arguments* and the parameters of *B* are called *formal arguments*.

- **Formal Parameter:** A variable and its type as they appear in the prototype of the function or method.
- **Actual Parameter:** The variable or expression corresponding to a formal parameter that appears in the function call in the calling environment.

In the above example, x, y and z in the main function are the actual parameter of add function. Formal parameter of add function are a, b and c.

PASS BY VALUE

In this parameter passing any changes or modification happened parameter won't reflect back to actual parameter. This can be of

```
#include<stdio.h>
int swap(int a,int b)
{
        int temp;
        temp=a;
        a= b;
        b=temp;
}
void main()
{
        int x,y;
        printf("Enter the numbers:");
        scanf("%d%d",&x,&y);
        printf("Before swapping : x=%d\ty=%d\n",x,y);
        swap(x,y);
        printf("After swapping : x=%d\ty=%d",x,y);
}
```

Output

Enter the numbers:10 20 Before
swapping: x=10 y=20After
swapping : x=10 y=20

In above program we expect the program to swap the value of x and y after calling the function swap with x and y as actual parameter. But swapping does not take place as this uses call by value as parameter passing technique.

During Swap call, a copy of actual parameters are created and changes are made on thatcopy. So the value of x and y does not changes.

After reading the value of x and y

x: 10	y: 20

During swap call a copy is created and passed to formal parameter

x: 10	a: 10

After the swap call, formal parameters get swapped but actual parameters remains thesame.

x: 10	a: 20
y: 20	b: 10

TYPES OF USER DEFINED FUNCTIONS

- Functions with no arguments and no return values
- Functions with arguments and no return values
- Functions with no arguments and but return a value
- Functions with arguments and one return values
- Function that have multiple return values(work based on condition)

- **Functions with no arguments and no return values**

// function to read two numbers and print its sum

```
void add()
{
        int a,b,sum;
        printf("Enter the values of a & b");
        scanf("%d%d",&a,&b);
        sum = a+b;
        printf("Sum=%d",sum);
}
```

- **<u>Functions with arguments and no return values</u>**

// function that takes two arguments and print their sum.

```
void add(int a,int b)
{
        int sum;
        sum= a + b;
        printf("Sum=%d",sum);
}
```

- **<u>Functions with no arguments and return one value</u>**

// function to read two numbers and return its sum

```
int add()
{
        int a,b,sum;
        printf("Enter the values of a & b");
        scanf("%d%d",&a,&b);
        sum = a+b;
        return sum;
}
```

- **<u>Functions with arguments and one return value</u>**

// function that takes two arguments and print their sum.

```
int add(int a,int b)
{
        return (a + b);
}
```

<u>Function that have multiple return values</u>

```
#include <stdio.h>
int even(num)
 {
          if( num%2==0)
                  return 1;
         else
          }
 return 0;
        void main()
        {
                int s,a;
                printf("enter a");
                scanf("%d",&a);
                printf("%d",even(a));
        }
```

- **Write a program to perform arithmetic operations using function**

```
#include<stdio.h>
int add(int a,int b)
{
}
int diff(int a,int b)
{
    return (a-b);
```

```
}
int mul(int a,int b)
{
    return (a*b);
}
float div(int a,int b)
{
    return (float)a/b;
}
int mod(int a,int b)
{
    return (a%b);
}
void main()
{
    int x,y;
    printf("Enter the values:");
    scanf("%d%d",&x,&y);
    printf("Sum=%d\n",add(x,y));
    printf("Difference=%d\n",diff(x,y));
    printf("Multiply=%d\n",mul(x,y));
    printf("Division=%f\n",div(x,y));
    printf("Modulo=%d\n",mod(x,y));
}
```

OUTPUT

```
Enter the values:21 8
Sum=29
Difference=13
Multiply=168
Division=2.625000
Modulo=5
```

- **Write a program to display prime numbers upto a range using function.**

```
// function takes an argument n, if n is prime it will return 1 otherwise 0

#include<stdio.h>
#include<math.h> int
checkPrime(int n)
{
    int flag=0,i;
    {
        if(n%i == 0)
        {
            flag =1;
            break;
        }
    }
    if(flag == 0)
        return 1;
else
}
  return 0;
    void main()
    {
        int n,i;
        printf("Enter the range:");
        scanf("%d",&n);
```

```
        for(i=2;i<=n;i++)
        {
                if(checkPrime(i) == 1)
                        printf("%d\t",i);
        }
    }
```

OUTPUT

```
Enter the range:25
2    3    5    7    11    13    17    19    23
```

RECURSIVE FUNCTION

In some problems, it may be natural to define the problem in terms of the problem itself. Recursion is useful for problems that can be represented by a simpler version of the same problem.

Example: the factorial function

6! = 6 * 5 * 4 * 3 * 2 * 1

We could write:

6! = 6 * 5!

A function that calls itself is known as a recursive function. And, this technique is known as recursion. While using recursion, programmers need to be careful to define a termination condition from the function; otherwise it will go into an infinite loop. Termination condition will be the base case where the problem can be solved without use loops instead as recursion is usually much slower.

- **Write a program to find factorial of a number using recursive function**

```
#include<stdio.h>
int fact(int n)
{
    if( n == 0)
            return 1;
else
}
  return n*fact(n-1);

    void main()
    {
        int n;
        printf("Enter the number:");
        scanf("%d",&n);
        printf("Factorial=%d",fact(n));
    }
```

OUTPUT

```
Enter the number:5
Factorial=120
```

- **Write a program to find nCr and nPr.**

```
#include<stdio.h>
int fact(int n)
{
    if( n == 0)
            return 1;
else
}
  return n*fact(n-1);

    void main()
    {
        int n,r;
        float C,P;
```